十字路口交通灯控制系统实验报告

Python 模拟 | main.py | 2026-09

1 方法介绍

1.1 设计的交通灯状态

系统采用“状态 + 持续时间 + 状态切换”的思路来模拟真实交通灯，共设计 4 个状态，每个状态对应南北、东西两个方向的灯色，并有固定的持续时长：

- 状态 1：南北绿灯、东西红灯，持续 5 秒
- 状态 2：南北黄灯、东西红灯，持续 2 秒
- 状态 3：南北红灯、东西绿灯，持续 5 秒
- 状态 4：南北红灯、东西黄灯，持续 2 秒

程序中状态用整数 1~4 表示，各状态对应的显示文本存放在字典 state_text 中；状态编号与持续时长的关系由函数 duration(state) 统一处理（状态 1、3 返回 5 秒，状态 2、4 返回 2 秒）。把“状态编号”和“显示文本”分开存放，做状态切换和倒计时时只操作数字，输出时再查表换成文字，代码更清晰。

1.2 不同状态之间如何切换

正常情况下，状态按 1 → 2 → 3 → 4 → 1 的顺序不断循环，切换关系如下：

```
状态1（南北绿灯/东西红灯，5秒）
  ↓
状态2（南北黄灯/东西红灯，2秒）
  ↓
状态3（南北红灯/东西绿灯，5秒）
  ↓
状态4（南北红灯/东西黄灯，2秒）
  ↓
状态1 → 循环
```

每个状态先显示自己的持续时长（每秒打印一次剩余时间），时间结束后调用状态切换函数得到下一个状态的编号，再进入下一轮循环，从而模拟真实交通灯控制器的连续运行。

1.3 change_state() 函数的作用

题目要求设计一个 change_state(state) 函数，作用是根据当前状态决定下一个状态。main.py 中对应的实现命名为 change(state)，功能与题目要求的 change_state() 完全一致，返回值为下一个状态的编号：

```
def change(state):   # 根据当前状态返回下一个状态：1→2→3→4→1
    return state % 4 + 1
```

函数体只有一行，利用取模运算实现循环：change(1)=2、change(2)=3、change(3)=4，而 change(4) 时 4 % 4 + 1 = 1，正好回到状态 1。相比用多条 if/elif 逐个判断，这种方式代码更简洁，也不容易漏掉边界情况。

1.4 while 循环在程序中的作用

程序中用了几处循环，各司其职：

- 外层 while True：让主程序永不退出，模拟真实交通灯控制器一直运行，是整个系统的“总开关”；
- 内层倒计时循环 for remain in range(duration(state), 0, -1)：控制当前状态持续规定秒数，每秒打印一次剩余时间；
- 紧急模式内部 while True：在紧急状态下保持南北、东西全红，并持续检查是否按下了 0。

其中外层 while True 的作用最关键：没有它，run() 执行一遍就结束了，交通灯无法自动连续循环，所以系统必须依赖这个永不退出的循环来模拟真实控制器。

1.5 紧急模式是如何加入正常控制流程的

紧急模式要求“无论当前处于什么状态，检测到 1 都立即进入全红，按 0 结束并回到状态 1”。加入时主要解决了“如何在不卡住倒计时的前提下检测按键”这个问题：

- 最初使用 input() 检测输入，但 input() 会阻塞程序，倒计时和状态切换全部停住；
- 尝试过 threading 后台线程方案，但输入 1 之后还需要再按回车，交互不够方便；
- 最终选用 Windows 下的 msvcrt 库：msvcrt.kbhit() 非阻塞地检查有没有按键，msvcrt.getch() 读取按键，不需要按回车。

具体实现上，把每 1 秒拆成 20 个 0.05 秒的小段，每段调用一次 check_key() 检测按键；一旦检测到 1，就把 emergency_mode 置为 True，并立即 break 当前的倒计时。主循环开头判断 emergency_mode，为真时进入紧急模式分支：打印“南北：红灯 / 东西：红灯”，然后在内部循环中等待按 0；按 0 后把 state 重置为 1、emergency_mode 置回 False，系统从状态 1 重新开始正常循环。等待按 0 的循环里还加了 time.sleep(0.1)，避免程序空转占用 CPU。

2 运行结果

运行环境：Windows 11，Python 3.14。直接执行 python main.py 得到以下结果（按实际运行顺序整理）。

2.1 正常交通灯循环运行结果

系统启动后从状态 1 开始，自动倒计时并切换状态，持续循环。以下是正常运行时的输出：

```
交通灯控制系统启动
提示：按 1 触发紧急模式，按 0 结束紧急模式
当前状态：南北绿灯,东西红灯，持续 5 秒
剩余时间：5 秒
剩余时间：4 秒
剩余时间：3 秒
剩余时间：2 秒
剩余时间：1 秒
切换交通灯...
当前状态：南北黄灯,东西红灯，持续 2 秒
剩余时间：2 秒
剩余时间：1 秒
切换交通灯...
当前状态：南北红灯,东西绿灯，持续 5 秒
剩余时间：5 秒
剩余时间：4 秒
剩余时间：3 秒
...
```

可以看到绿灯状态倒计时 5 秒、黄灯状态倒计时 2 秒，与题目规定一致；程序持续运行、自动切换，说明外层 while True 循环生效。

2.2 完整循环：状态1 → 状态2 → 状态3 → 状态4 → 状态1

一次完整循环的输出如下（共 14 秒，即 5 + 2 + 5 + 2）：

```
交通灯控制系统启动
提示：按 1 触发紧急模式，按 0 结束紧急模式
当前状态：南北绿灯,东西红灯，持续 5 秒
剩余时间：5 秒
剩余时间：4 秒
剩余时间：3 秒
剩余时间：2 秒
剩余时间：1 秒
切换交通灯...
当前状态：南北黄灯,东西红灯，持续 2 秒
剩余时间：2 秒
剩余时间：1 秒
切换交通灯...
当前状态：南北红灯,东西绿灯，持续 5 秒
剩余时间：5 秒
剩余时间：4 秒
剩余时间：3 秒
剩余时间：2 秒
剩余时间：1 秒
切换交通灯...
当前状态：南北红灯,东西黄灯，持续 2 秒
剩余时间：2 秒
剩余时间：1 秒
切换交通灯...
当前状态：南北绿灯,东西红灯，持续 5 秒
剩余时间：5 秒
剩余时间：4 秒
...
```

从输出可以看出，状态依次为状态1 → 状态2 → 状态3 → 状态4 → 状态1，其中绿灯（状态 1、3）倒计时 5 秒、黄灯（状态 2、4）倒计时 2 秒，最后回到状态 1
重新开始，验证了状态循环的正确性。

2.3 触发紧急模式后的运行结果

在运行过程中按 1，系统会立即中断当前倒计时并进入紧急模式；按 0 后结束紧急模式，从状态 1 重新开始。以下是在状态 1 的第 1 秒按下 1 时的运行结果：

```
交通灯控制系统启动
提示：按 1 触发紧急模式，按 0 结束紧急模式
当前状态：南北绿灯,东西红灯，持续 5 秒
剩余时间：5 秒
！！！检测到紧急情况！！！
南北：红灯
东西：红灯
交通灯系统进入紧急模式
（按 0 结束紧急模式）
紧急模式结束，交通灯系统恢复正常运行
切换交通灯...
当前状态：南北绿灯,东西红灯，持续 5 秒
剩余时间：5 秒
剩余时间：4 秒
剩余时间：3 秒
剩余时间：2 秒
剩余时间：1 秒
切换交通灯...
当前状态：南北黄灯,东西红灯，持续 2 秒
剩余时间：2 秒
剩余时间：1 秒
...
```

可以看到，在状态 1 的倒计时中按下 1 后，程序立即输出“！！！检测到紧急情况！！！”，南北、东西两个方向都变为红灯并进入紧急模式；按 0 后输出“紧急模式结束”，系统重新从状态 1（南北绿灯、东西红灯）开始运行。由于程序每 0.05 秒就检测一次按键，无论当前处于哪个状态，都能很快响应。

3 总结

本实验用 Python 模拟了十字路口交通灯控制系统：通过“状态 + 持续时间 + 状态切换函数”实现 4 个状态的自动循环，利用 while 循环让系统持续运行，并使用 msvcrt 非阻塞按键检测把紧急模式加入正常控制流程。程序实测满足题目要求：正常循环时绿灯 5 秒、黄灯 2 秒；任意时刻按 1 立即进入全红紧急模式，按 0 后从状态 1 重新开始。